

Understanding Transformer Reasoning Capabilities via Graph Algorithms

Depth is parallel rounds; the hierarchy is the contribution

Sanford, Fatemi, Hall, Tsitsulin, Kazemi, Halcrow, Perozzi, Mirrokni (Google Research · Columbia · DeepMind) ·
NeurIPS 2024 · arXiv 2405.18512v1

Reading companion · GNN / Graph-Transformer thesis exploration

★ TL;DR

The paper’s object is not a model but a **hierarchy**. It sorts 9 graph reasoning tasks into three families — **retrieval**, **parallelizable**, **search** — and matches each to the smallest transformer **scaling regime** that can solve it. The engine is Theorem 1: an R -round **MPC** (massively parallel computation) protocol with $O(N^\delta)$ local memory is simulated by a transformer of depth $L = O(R)$ and embedding dimension $m = O(N^{\delta+\epsilon})$ — **one layer buys one round of parallel communication**. Connectivity is L -complete and $O(1)$ -round-equivalent to every other parallelizable task, so it lands in **LogDepth**: $L = O(\log N)$, $m = O(N^\epsilon)$. Conversely, one layer provably cannot do it ($mH = \tilde{\Omega}(N)$). Empirically they train small transformers **from scratch** on GraphQA and beat both GNNs and prompted LLMs on global tasks — while **losing to GNNs on local tasks in the low-sample regime**.

♦ The big question

Which transformer **scaling regimes** — depth L , width m , number of blank “pause” tokens N' — perfectly solve which classes of algorithmic graph problems?

♦ The one-line answer

Depth tracks **parallel round complexity**. Retrieval needs one layer; parallelizable tasks need $\Theta(\log N)$; search tasks need log depth **and** much larger width.

1 · Setup — the tokenization we copied

► Definition — Graph encoding (their Fig. 1)

A graph $G = (V, E)$ becomes a sequence of length $N = O(|V| + |E|)$: **vertex tokens** $v_1 \dots v_n$, then **edge tokens** (v_i, v_j) — one token per edge — then a **task token** (e.g. the pair to test). Output is read off the final position. A standard decoder-free transformer stack processes it.

📌 My note

This is exactly our `node_edge` tokenization (`docs/CONFIG.md`), so the paper is the direct source for that mode. Two consequences worth keeping straight: (i) N in every bound below is the **sequence length** $O(|V| + |E|)$, **not** the node count — a bound of $O(\log N)$ on a dense graph is $O(\log n^2) = O(2 \log n)$, i.e. still logarithmic in n but with the constant hiding a factor 2; (ii) because edges are first-class tokens, edge count changes sequence length — the exact channel that became our **serialization leak** (`RESEARCH-QUESTIONS.md` A4, Leak 3). Their setup has that surface too.

The four scaling regimes (§3)

Regime	depth L	width m	pause tokens N'
Depth1 (D1)	1	$O(\log N)$	none
LogDepth (LD)	$O(\log N)$	$O(N^\epsilon)$	none
LogDepthPause (LDP)	$O(\log N)$	$O(N^\epsilon)$	$\text{poly}(N)$ blank

Regime	depth L	width m	pause tokens N'
LogDepthWide (LDW)	$O(\log N)$	$O(N^{1/2+\varepsilon})$	none

✓ A6 — resolved exactly

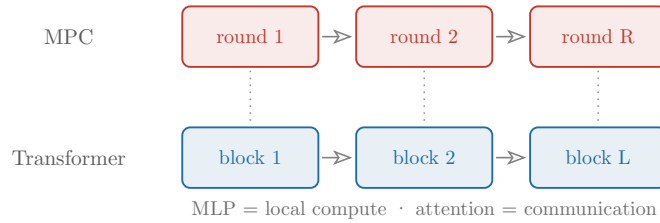
Your recollection “**connectivity is parallelizable, needing $L = O(\log N)$ and $m = O(N^\varepsilon)$** ” is the verbatim definition of the **LogDepth** regime, and connectivity is classed LD (their Fig. 2b). Cite it as: Sanford et al. 2024, §3, LogDepth regime; the underlying simulation is Theorem 1. Note $\varepsilon > 0$ is **any fixed constant** — the claim is sub-polynomial width, not a specific exponent.

2 · The engine — depth $\leftrightarrow$ MPC rounds

◆ Theorem — 1 (MPC simulation, simplified)

For constants $\delta, \varepsilon > 0$: any R -round MPC protocol with N machines and $O(N^\delta)$ bits of local memory each can be simulated by a transformer of depth $L = O(R)$ and embedding dimension $m = O(N^{\delta+\varepsilon})$.

The mechanism (Appendix B): **local computation** $\rightarrow$ the element-wise MLPs; **communication** $\rightarrow$ a single multi-headed self-attention layer. One MPC round = one transformer block.



💡 Thesis hook

This is the sentence the whole thesis rests on, and it is worth stating in its **strong** form: depth is not a capacity knob, it is a **round budget**. Our `reports/message-passing.typ` makes the same argument geometrically (additive 1-hop reach vs. the doubling trick); this theorem is the formal version, and it is the citation to use when the argument needs authority rather than intuition.

3 · The hierarchy

◆ Theorem — 2 (parallelizable $\subseteq$ LDP, LDW)

For any **parallelizable** task there exist transformers in **LogDepthPause** and **LogDepthWide** that solve it.

◆ Theorem — 3 (log depth is necessary — conditional)

Conditional on their Conjecture 13, any transformer solving **some** parallelizable task with width $mH = O(N^\varepsilon)$ and $N' = \text{poly}(N)$ pause tokens must have depth $L = \Omega(\log N)$.

◆ Theorem — 4 (search $\subseteq$ LDW)

For any **search** task there exists a transformer in **LogDepthWide** that solves it.

◆ Theorem — 5 (retrieval $\subseteq$ D1)

For any **retrieval** task — node count, edge count, edge existence, node degree — there exists a **single-layer** transformer with $m = O(\log N)$ that solves it.

◆ Theorem — 6 (one layer is not enough for the real tasks)

Any **single-layer** transformer solving graph connectivity, shortest path, or cycle detection has width $mH = \tilde{\Omega}(N)$ — i.e. width must blow up to sequence-length scale.

Family	Example tasks	Regime
Retrieval	node count, edge count, edge existence, node degree	D1
Parallelizable	connectivity , cycle check, bipartiteness, MSF, planarity, #components	LD
Search	shortest path, diameter, <i>st</i> -reachability, radius, median	LDW

✍ My note

The unifying trick is **$O(1)$ -round MPC equivalence**: all parallelizable tasks are interreducible in constant rounds, so a single result about connectivity transfers to the whole family at no extra depth. Same for search tasks and shortest path. That is why the hierarchy is only three rows wide rather than nine.

✍ My note

Erratum. Theorem 2 as printed reads “there exists transformers in LogDepthPause and LogDepthPause”. From the following paragraph (which derives one bound from Theorem 9 and the other from Theorem 15) the second is plainly **LogDepthWide**. Harmless typo, but do not quote the sentence verbatim.

§3.4 — the one place sub-logarithmic depth appears

◆ Theorem — 7 (triangle counting)

Triangle counting on graphs of arboricity α is solved with $m = O(N^\epsilon)$, depth $L = O(\log \log N)$, and $N' = O(\alpha N^{1-\epsilon})$ pause tokens if $\alpha = \Omega(N^\epsilon)$, else none.

✍ My note

Worth noticing as the exception that proves the rule: **only here** does depth drop below log, and the price is pause tokens. Unlike the parallelizable and search classes, strictly sub-logarithmic depth is attainable **even for worst-case graphs** — but the computation has been moved onto the tape rather than into depth. That is the same depth-for-length exchange our AR-CoT results exploit (Q5).

4 · Experiments — and the A7 correction

They evaluate on **GraphQA**: small autoregressive transformers (up to 60M params) trained **from scratch**, plus a **fine-tuned T5-11B**, against GCN / MPNN / GIN and against **prompted LLMs** (zero-shot, few-shot, CoT, CoT-BAG).

✓ A7 — the memory was inverted

The claim “**they used an LLM for the graph task execution**” is backwards as an account of the paper’s method. §4.3 is titled “**Trained transformers outperform LLM prompting**”: the LLM prompting results are a **baseline they beat**, not their approach. On connectivity, their 60M-100K transformer scores **98.0** against the best prompting number **84.9** (zero-shot); on shortest path **97.2** vs **38.6** (CoT). The fine-tuned 11B model is a **trained** model, not a prompted one.

Model	Connectivity	Shortest path	Node degree	Cycle check
best LLM prompting	84.9	38.6	29.2	76.0
GCN	83.8	55.0	9.4	83.2

Model	Connectivity	Shortest path	Node degree	Cycle check
MPNN	94.4	72.6	99.8	100.0
GIN	94.0	58.6	37.8	83.2
60M transformer (100K)	98.0	97.1	91.7	98.0
11B transformer (FT, 1K)	98.4	92.8	68.8	98.0

✍ My note

Do not summarise this as “transformers beat GNNs”. The paper’s own §4.2 says the opposite for local tasks: **MPNN beats the 60M transformer on node degree (99.8 vs 91.7) and cycle check (100.0 vs 98.0)**, and GNNs dominate in the **low-sample** (1K) regime across the board. Their claim is precisely scoped: transformers win on tasks needing **global** aggregation (connectivity, shortest path); GNNs win where a **local** inductive bias is correct and samples are scarce. That scoping is the honest version and it is also the more useful one for us — it is an empirical echo of the reach argument.

5 · Two caveats that matter for our datasets

? Open question

Their empirical graphs are small, dense ER. GraphQA: 1,000 train / 500 dev / 500 test, Erdős–Rényi, **5–20 nodes**, avg. degree **5.43**, avg. 37 edges. That is the **same generator family** our own leak audit convicted (RESEARCH-QUESTIONS.md A4, Leak 2: ER near threshold makes “min degree ≥ 1 ” a 98.2% rule). Their tasks differ enough that the degree-0 shortcut may not transfer directly — but **nobody in the paper audits for it**, and the empirical section is where the hierarchy is claimed to be validated. This is a concrete, defensible gap to point at, and it is our Q2 applied to the reference paper.

? Open question

The paper says so itself. Appendix E notes the GraphQA graphs “have very small cycles (Figure 7) and do not resemble the large-diameter worst-case instance” that the negative results (Theorems 3, 6) are actually about. So the **theory** is worst-case and the **experiments** are small-diameter average-case — the two halves never meet on the same instances. Our `connectedness_hard_diam` is built exactly to span that gap.

✍ My note

A terminology trap. GraphQA’s “connectivity” is an **edge-level** task — “is there a path from node u to node v ” (*st*-connectivity) — not our **graph-level** “is the whole graph connected”. Our binary `connectedness_hard` label is a conjunction over all pairs, so it is strictly a **harder read-off** than their target even on the same graphs. Worth one sentence in the thesis so a committee does not think the tasks are identical.

💡 Thesis hook

Sample-complexity ablations (E.4.1) are directly comparable to our capacity control. They train to 1,000,000 steps across sample sizes and report: edge/node count solved at **100 samples**; connectivity, shortest path, node degree “perfectly fit the training set in most sample size regimes” but only close the train–test gap at **100,000**; cycle check and triangle count **never** close it. That is the same shape as our `overfit10` $\rightarrow$ 1.00 / `overfit100` $\rightarrow$ 0.76 / `full` $\rightarrow$ 0.545 control (A8) — fitting is easy, generalizing is the wall. Their curve is the published precedent for our framing.

6 · Hooks into our work

💡 Thesis hook

1. **This is the citation for Q1’s premise.** Q1 asks whether SGD **finds** the log-depth construction. Theorems 2–3 establish that the construction **exists** and that log depth is (conditionally) **necessary** — so the representability half of the expressivity/learnability gap is fully sourced here, and our contribution is cleanly the **learnability** half.

💡 Thesis hook

2. **Their D1 result explains our node_edge failure — partly.** Theorem 6 says a single-layer transformer needs $mH = \tilde{\Omega}(N)$ for connectivity. Our `node_edge` runs pinned at $\ln 2$ were depth-limited **and** narrow, so the theory predicts failure. But careful: our diagnosed mechanism was the **learned node-id gradient cancellation** (A8), which is a trainability failure, not a width bound. Two different reasons for the same number — do not merge them in the write-up.

? Open question

3. **The width claim is now cited but still untested by us.** We have never run the $m = O(N^\epsilon)$ axis: our depth×width grid (A8) tops out at width 256 with N fixed. Testing the regime honestly means scaling m **with** N and checking whether the transition predicted by LD appears. That is a real experiment and nobody in the repo has done it.

7 · One-paragraph summary

Sanford et al. give the field a **complexity-theoretic map** of transformer graph reasoning: depth simulates parallel rounds (Thm 1), so tasks sort by **round complexity** rather than by apparent difficulty. Retrieval is one layer (Thm 5); connectivity and its $O(1)$ -round equivalents need logarithmic depth with sub-polynomial width, and one layer provably cannot do it (Thms 2, 3, 6); search tasks need logarithmic depth with much larger width (Thm 4). Empirically, small from-scratch transformers on GraphQA beat GNNs and prompted LLMs on **global** tasks while **losing to MPNNs on local ones in low-sample regimes**. The gap they leave open — and the one our thesis lives in — is that all of this is **representational**: they never establish that gradient descent finds these constructions on data without statistical shortcuts, and their own benchmark is small-diameter ER that the paper admits does not resemble the worst case its negative results describe.

PDF: [papers/sanford-2024-transformer-reasoning-graph-algorithms.pdf](#) · [arXiv:2405.18512v1](#) · Resolves [RESEARCH-QUESTIONS.md](#) A6 and A7 · Related in this repo: [message-passing.typ](#), [ye-2026-notes.typ](#), [executing-graph-algorithms.md](#)